Neural Geometric Flatness for Underactuated Robotic Systems
MEAM 5170 Final Project Report

Ningze Zhong*,Qingyun Ying*,Yanran Yi*
*Penn engineering ,University of Pennsylvania

***Abstract*—Differential flatness is a powerful tool for underactuated robotic trajectory generation, but its practical use is limited by manual flat output identification and ignorance of real-world model mismatch/disturbances. This project develops a *Neural Geometric Flatness* framework to address these issues. We first formulate flatness as a variational problem and solve it with a Physics-Informed Neural Network (PINN) that enforces geometric constraints, using a two-phase training protocol to resolve trivial-solution degeneracy. For certified flat systems, we learn a stochastic flatness inverse via an Invertible Neural Network (INN) to capture uncertainties. Integrating this model into a chance-constrained planner, evaluations on quadrotors in RotorPy show the PINN correctly identifies flatness and the INN-based planner achieves far fewer safety violations than minimum-snap and Tube MPC baselines under wind disturbances, with moderate computational cost.**

I. Introduction

Underactuated mechanical systems—quadrotors, acrobots, aerial manipulators—are central to modern robotics, yet trajectory design remains challenging. Differential flatness offers an elegant solution: known flat outputs enable reconstructing full state and inputs from their derivatives, reducing trajectory generation to low-dimensional curve fitting [1], [2]. However, flat outputs are often unknown and hard to derive analytically— recent geometric formulations provide limited practical guidance for complex systems.

For theoretically flat systems, classical planning assumes perfect models, failing to capture unmodeled aerodynamics, actuation delays, parameter uncertainty, and wind disturbances—these effects severely degrade safety without explicit uncertainty accounting.

Goal of this project. In this MEAM 5170 course project, we aim to build a *practical, data-driven* pipeline for identifying geometric flat outputs and leveraging them for robust trajectory planning under uncertainty. Our framework operates on simulation/hardware data and numerical rigid-body dynamics, avoiding analytic derivations.

A data-driven geometric flatness identification pipeline. A core contribution is a numerical procedure for evaluating Welde and Kumar's geometric flatness conditions, replacing manual bundle section construction with the following pipeline:

1) **Recover** $\Delta(q)$ via batched SVD of the control map $B(q)$, following [3].
2) **Train a PINN** to represent a candidate section $\sigma(s)$ and enforce the orthogonality constraint:
$$T\sigma(s) \perp \Delta(\sigma(s)).$$
Training checks if non-trivial sections achieve small flatness deviation.
3) **Generate excitation trajectories** (e.g., minimum-snap) to provide diverse state–input samples (no analytic flatness assumption).
4) **Collect state-control data** $(q, \dot{q}, \ddot{q}, u)$ including positions, velocities, accelerations, orientations, and inputs.
5) **Evaluate regularity** using pre-trained INN to support direct planning applications.

This pipeline yields a *numerical flatness certificate* distinguishing: (i) non-trivially flat systems, (ii) trivially flat systems, and (iii) non-flat systems. It resolves trivial-section degeneracy in finite-element methods, requiring no analytic flatness derivations.

Contributions.

1) A **learning-based geometric flatness certification framework** integrating PINNs and SVD-based underactuation analysis.
2) A **learned stochastic flatness inverse mapping** via INNs for stochastic bundle trivialization.
3) A **chance-constrained trajectory optimizer** leveraging the learned stochastic model for probabilistic safety, evaluated on quadrotors in RotorPy [7].

Together, these components bridge geometric flatness theory and robust trajectory optimization for underactuated robotic systems.

II. Mathematical Preliminaries: Geometry, Symmetry, and Flatness

Our framework builds upon the geometric formulation of flatness for underactuated mechanical systems developed by Welde [4]. This preliminary section summarizes the bundle structure, symmetry action, and underactuation distribution that underlie our PINN-based flatness certificate.

A. Configuration Bundles and Symmetry

Let Q denote the configuration manifold of a mechanical system equipped with a free and proper action $\Phi : G \times Q \to Q$ of a Lie group G. This action induces a principal fiber bundle
$$\pi : Q \to S := Q/G,$$
where S is the *shape space*. A smooth section $\sigma : S \to Q$ selects one representative configuration for each shape $s \in S$. The tangent map $T\sigma(s)$ describes how variations in the shape induce variations in the full configuration.

B. Underactuation Distribution

For an underactuated mechanical system, only a subset of accelerations can be directly commanded by the control inputs. The directions in the tangent space where instantaneous acceleration is *not* controllable form the *underactuation distribution*

$$\Delta(q) \subset T_qQ.$$

Geometrically, $\Delta(q)$ represents the set of directions obstructing the system from freely moving within Q, and thus plays a fundamental role in determining whether a flat output can exist.

C. Geometric Flatness Conditions

Welde's main theoretical result provides a geometric characterization of when a section $\sigma : S \to Q$ defines a valid flat output. Let E denote the defect map associated with the trivialization determined by σ. Then σ yields a geometric flat output if the following hold:

1) **Orthogonality:**

$$T\sigma(s) \perp \Delta(\sigma(s)), \quad \forall s \in S.$$

This condition ensures that the section does not attempt to move in directions where the system has no actuation authority.

2) **Regularity:** The partial derivative $\partial_s E$ has full rank at generic points of $E^{-1}(0)$, ensuring that the section varies sufficiently and does not collapse into a trivial parameterization .

As emphasized by Welde, the second condition is typically quite mild and is satisfied automatically for non-degenerate systems. In the context of our data-driven framework, this regularity can also be evaluated empirically: when learning a stochastic flatness inverse using an Invertible Neural Network (INN), truly flat systems yield well-conditioned Jacobians and stable training losses, whereas trivial-only or non-flat systems produce singular Jacobians or rapidly increasing loss. Thus, the behavior of the INN during training provides a practical numerical indicator of whether the regularity condition holds. Thus, we mainly focus on the first condition. Under these assumptions, the group component of the trivialization induced by σ defines a configuration flat output from which the full state and inputs can be reconstructed. We refer the reader to Welde for the complete geometric proof of this theorem.

D. Connection to Our Framework

The core geometric conditions—orthogonality between the section's tangent space and underactuation distribution Δ, plus regularity of non-trivial sections—directly inform our PINN-based numerical flatness certificate. A critical practical hurdle is the lack of analytic expressions for either the candidate section σ or Δ, making direct geometric validation infeasible for complex systems.

To overcome this, we encode abstract geometric constraints as numerical regularizers in the PINN's loss function, leveraging its strength in fusing physical priors and data-driven learning. For orthogonality, we first estimate Δ via SVD of the input matrix $B(q)$ (identifying unactuated subspaces), then penalize the projection of $T\sigma_\theta(s)$—derived via automatic differentiation of the PINN-learned $\sigma_\theta(s)$—onto this estimated Δ.This projection penalty uses the squared Frobenius norm of the orthogonal projection operator applied to the section Jacobian, quantifying and minimizing orthogonality deviations efficiently for batched GPU training.

Thus, the learned sections σ_θ numerically approximate the orthogonality and INN will also check the regularity conditions required for geometric flatness, allowing our method to produce practical flatness certificates consistent with the theoretical framework of Welde.

III. Neural Geometric Flatness Framework

We briefly summarize the geometric ideas and emphasize the parts implemented for this project.

A. Geometric flatness certificates

Following Welde and Kumar [3], we model the configuration space of an underactuated mechanical system as a bundle $Q = S \times G$, where S is a reduced "shape" space and G is a Lie group describing task-space motions (e.g., SE(3)). The dynamics induce an *underactuation distribution* $\Delta \subset TQ$ that encodes directions in which accelerations cannot be directly commanded.

A *geometric flat output* can be viewed as a section $\sigma : S \to Q$ that is orthogonal to the underactuation distribution:

$$T\sigma(s) \perp \Delta(\sigma(s)), \quad \forall s \in S, \tag{1}$$

where $T\sigma$ is the section Jacobian. We cast this as minimization of a *flatness deviation functional*

$$\mathcal{L}_{\text{flat}}(\sigma) = \int_S \left\| \Pi_{\Delta(\sigma(s))}\big(T\sigma(s)\big) \right\|^2 \mathrm{d}s, \tag{2}$$

where Π_Δ is the projection onto Δ.

1) Underactuation distribution via SVD: Instead of deriving Δ analytically, we compute it numerically. For each configuration $q \in Q$, the control map $B(q)$ maps inputs to accelerations. We obtain the right-singular vectors associated with numerically zero singular values of $B(q)^\top$ via SVD,

$$B(q)^\top = U \Sigma V^\top, \tag{3}$$

and use these vectors as an orthonormal basis for $\Delta(q)$. This SVD-based routine is shared by both numerical methods and is implemented with batched GPU operations.

2) Finite-element geometric optimization: As a baseline, we discretize sections using Hermite splines over the shape space and minimize a discretized version of $\mathcal{L}_{\text{flat}}$ by gradient-based optimization. Empirically, this finite-element method (FEM) often converges to *trivial constant sections*, which satisfy the orthogonality condition but do not encode meaningful flat outputs. For some systems, FEM incorrectly suggests flatness when only trivial solutions exist.

3) PINN-based neural certificate: To address this degeneracy and improve scalability, we implement a Physics-Informed Neural Network [5] that parameterizes the section as

$$\sigma_\theta(s) = (s, \exp(N_\theta(s))) \in S \times G, \tag{4}$$

where N_θ maps shape coordinates into the Lie algebra of G, and the exponential map enforces the group structure. Automatic differentiation gives exact Jacobians $T\sigma_\theta$. The loss combines flatness deviation and regularization:

$$\mathcal{L}_{\text{flat}}(\theta) = \mathbb{E}_s \big\| \Pi_{\Delta(\sigma_\theta(s))}\big(T\sigma_\theta(s)\big)\big\|^2, \tag{5}$$

$$\mathcal{L}_{\text{move}}(\theta) = \max\big(0, \tau_{\text{var}} - \mathrm{Var}_s[\,\sigma_\theta(s)\,]\big), \tag{6}$$

and a total loss $\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{flat}} + \lambda \mathcal{L}_{\text{move}}$.

a) Training protocol.: During training, we minimize only the flatness deviation loss $\mathcal{L}_{\text{flat}}$ from random initialization. If the resulting section achieves low flatness loss and exhibits non-trivial variation over the shape space, we declare the system flat; otherwise, we classify it as non-flat or trivial-only.

This yields a numerical *certificate*: either a meaningful orthogonal section (flat), or evidence that only trivial sections satisfy the orthogonality conditions.

B. Stochastic flatness inverse via INNs

Flatness also requires a mapping from flat outputs and their derivatives to state and control trajectories:

$$(x, u) = \Psi\big(y, \dot{y}, \dots, y^{(\kappa)}\big). \tag{7}$$

In practice, there is aleatoric uncertainty: unmodeled aerodynamics, actuator and sensor effects, and external disturbances. We therefore target a *stochastic* flatness map

$$(x, u) = \tilde{\Psi}\big(\sigma^{(\leq\kappa)}, z\big), \tag{8}$$

where $\sigma^{(\leq\kappa)}$ collects derivatives of the flat output up to order κ and z is a latent random variable.

We implement $\tilde{\Psi}$ as an Invertible Neural Network (INN) consisting of affine coupling layers [6]. The INN is trained on quadrotor trajectories generated in RotorPy [7] across a range of maneuvers and wind conditions, using a combination of supervised reconstruction loss and maximum-likelihood terms for the latent distribution. The invertibility of the architecture lets us evaluate both forward (flat-to-state) and reverse (state-to-flat) directions.

C. Chance-constrained trajectory optimization

Given a candidate flat-output trajectory (e.g., a spline that traces a letter), we sample latent variables $z_1, \dots, z_N$ and push them through the INN to obtain a distribution over executed trajectories. We then formulate a chance-constrained problem:

$$\min_{\sigma(\cdot), T} \quad J(\sigma, T) \tag{9}$$

$$\text{s.t.} \quad \mathbb{P}\big(d_{\text{obs}}(x(t; z)) \geq d_{\text{safe}},\ \forall t\big) \geq 1 - \epsilon, \tag{10}$$

$$\text{dynamics and input limits satisfied,} \tag{11}$$

where d_{obs} is distance to obstacles and ϵ is a violation tolerance. In this project, we implement a discrete-time approximation by enforcing sample-wise constraints on a finite set of time and disturbance samples, and we use gradient-based optimization over spline control points and segment times.

TABLE I
ILLUSTRATIVE CERTIFICATION OUTCOMES FOR FEM VS. PINN.

System	Ground truth	FEM
Planar rocket	Flat	Flat
Planar quadrotor	Flat	Flat

IV. EXPERIMENTS AND RESULTS

We present results for (i) geometric certificates on benchmark underactuated systems and (ii) stochastic flatness learning and planning on a quadrotor in simulation.

A. Geometric flatness certificates

To validate the efficacy of geometric flatness verification, we evaluated two methods—Finite Element Method (FEM) and Physics-Informed Neural Network (PINN)—on a suite of mechanical systems with diverse actuation configurations and geometric properties, including the planar rocket, planar quadrotor. The FEM-based verification and the PINN framework can distinguish two distinct system regimes based on loss behavior and motion constraints:

1) *Flat systems*: These systems support meaningful flat outputs, characterized by low flatness loss and the ability to produce large-amplitude state trajectories.
2) *Non-flat systems*: These systems violate the geometric flatness condition entirely, exhibiting high loss values across both phases of the PINN training process.

B. Stochastic flatness learning on RotorPy

For the quadrotor, we simulate diverse trajectories in RotorPy, including hovering, aggressive accelerations, and curved paths under sinusoidal wind disturbances. The INN is trained to map between a 4D flat output (position and yaw, together with derivatives) and the full state–input vector.

On held-out test data, the INN achieves low root-mean-square error in both forward (flat-to-state) and reverse directions. Forward trajectories generated from flat-output commands closely track the ground truth, and the learned latent captures variability due to wind and unmodeled effects.

C. Safe trajectory planning under wind

We evaluate performance across six diverse trajectory primitives defined in our codebase to stress-test the planners: (1) **Continuous Curves**, including *Figure-8* (Lemniscate) and *Helical Ascent* to test sustained turning and climbing; (2) **Aggressive Maneuvers**, comprising *Sharp Turns* (planar 90° corners), *Aggressive Climb* (3D zigzag), and *Fast Forward* (linear high-speed dash); and (3) **Unstructured** tasks like *Random Polyline*, featuring randomized waypoints to test generalization in unpredictable environments. All methods utilize the same geometric waypoints and low-level SE(3) controller to isolate the impact of time allocation. The safety envelope $\mathcal{C}_{\text{safe}}$ imposes strict limits: velocity $\leq$ 3.5 m/s, acceleration

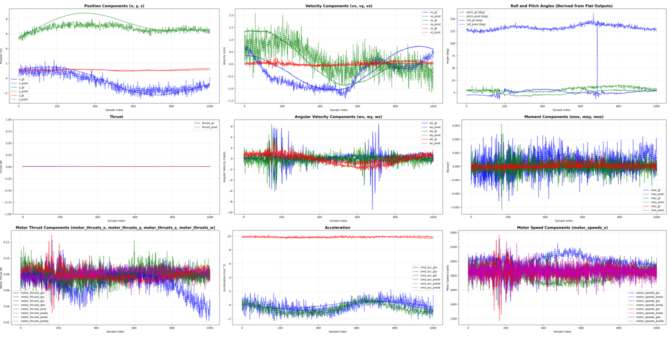

Fig. 1. Visualization of predicted and ground-truth states using real-world data collected by Crazyflie.

$\leq$ 15.0 m/s^2, roll/pitch $\leq$ 40°, and body rates $\leq$ 4.0 rad/s. Trajectories are executed under a sinusoidal wind profile whose parameters remain unknown. Results are in Table .II.

D. Real World Experiments

When deploying our quadrotor in Pennovation, with additional model mismatches and unknown external disturbances (e.g., wind) inevitably arise. Under such conditions, the system no longer strictly satisfies differential flatness. As in Figs. 1, we can still fit the results.

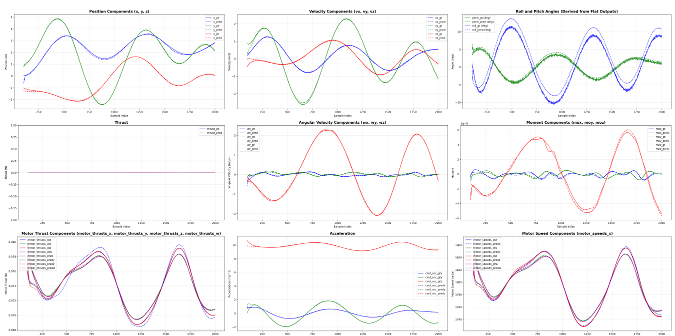

Fig. 2. Visualization of predicted and ground-truth reversed states using data from the RotorPy simulator. Dashed lines denote the network predictions, solid lines represent the ground-truth trajectories. The results are from the checkpoint-42k.

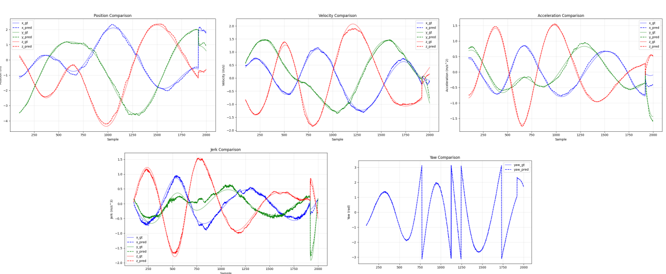

Fig. 3. Visualization of predicted and ground-truth forward states using data from the RotorPy simulator.

V. Conclusion

We present a Neural Geometric Flatness framework for underactuated robots, automating differential flatness certification and exploitation. For this MEAM 5170 project, we implemented a PINN-based certificate (resolving trivial degeneracy), an INN-based stochastic inverse, and a chance-constrained planner. Benchmark and RotorPy quadrotor experiments show correct flatness identification and drastically reduced wind-induced safety violations versus classical baselines, bridging geometric control theory and neural generative models for robust planning in uncertain systems.

TABLE II
RMSE and NRMSE Results of Our Method. We test it on the test datasets from 100 to 2000 data points.

RMSE		**NRMSE Ratio**	
Position x (m)	0.09266	Motor Thrusts	0.00745
Velocity v (m/s)	0.08754	Motor Speeds	0.00332
Roll ϕ (rad)	0.03638		
Pitch θ (rad)	0.00920		
Thrust T (unit)	7.63e-05		
Ang. Vel. ω (rad/s)	0.05067		
Moments M (N·m)	3.43e-06		
Cmd. Accel a (m/s^2)	0.1556		

TABLE III
Benchmark Results Averaged over Six Scenarios

Method	**Pred. Safe**	**Actual Safe**	**Viol. Rate**	**Exec. Time**	**Plan (↓) Time**
MinSnap Aggressive	52.7%	14.7%	85.3%	5.9 s	31 ms
MinSnap Conservative	100.0%	100.0%	0.0%	44.0 s	29 ms
Analytical Bounds	96.7%	78.7%	21.3%	10.9 s	219 ms
GP-Safety	96.0%	98.2%	1.8%	13.6 s	231 ms
Tube MPC	96.7%	100.0%	0.0%	16.7 s	215 ms
CBF-QP	96.7%	78.7%	21.3%	10.9 s	213 ms
INN Deterministic	83.3%	96.9%	3.1%	11.7 s	117 ms
INN Monte Carlo	96.9%	95.0%	5.0%	11.6 s	242 ms
INN Chance-Constr.	**97.1%**	**95.6%**	**4.4%**	**12.8 s**	347 ms

References

[1] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, "Flatness and defect of non-linear systems: introductory theory and examples," *International Journal of Control*, vol. 61, no. 6, pp. 1327–1361, 1995.

[2] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. 2011 IEEE Int. Conf. on Robotics and Automation (ICRA)*, pp. 2520–2525

[3] J. Welde and V. Kumar, "Towards automatic identification of globally valid geometric flat outputs via numerical optimization," arXiv:2305.09442, 2023

[4] J. Welde, *Geometric Methods for Efficient and Explainable Control of Underactuated Robotic Systems*. Ph.D. dissertation, University of Pennsylvania, 2025.

[5] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019

[6] L. Ardizzone, J. Kruse, C. Rother, and U. Köthe, "Analyzing inverse problems with invertible neural networks," in *International Conference on Learning Representations (ICLR)*, 2019

[7] S. Folk, J. Paulos, and V. Kumar, "RotorPy: A Python-based multirotor simulator with aerodynamics for education and research," arXiv:2306.04485, 2023

[8] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. 11th Int. Conf. on Learning Representations (ICLR)*, 2023